Classiq | Track C

Solving Linear Differential Equations

Team: 5th Harmony Bloch Busters

A Quantum Algorithm for Solving Linear Differential Equations (Tao Xin et al. 2020)

LDE:

$$\frac{dx(t)}{dt} = Mx(t) + b$$

Analytical Solution:

$$x(t) = e^{Mt}x(0) + (e^{Mt} - I)M^{-1}b.$$

k-th Order Solution:

$$x(t) \approx \sum_{m=0}^{k} \frac{(Mt)^m}{m!} x(0) + \sum_{n=1}^{k} \frac{M^{n-1}t^n}{n!} b,$$

A Quantum Algorithm for Solving Linear Differential Equations (Tao Xin et al. 2020)

1. Encoding

- Encode x(0) and b into circuit
- Calculate matrix operator V

$$V = \frac{1}{N}\begin{bmatrix} C & D \\ D & -C \end{bmatrix}$$ where $$C_m = \frac{\|x(0)\|(t)^m}{m!}, \quad D_n = \frac{\|b\|(t)^{n-1}t}{n!},$$

2. Entanglement

- Apply unitary matrix
- Entangle work qubits and ancillas

3. Decoding

- Reverse encoding operations
- Interfere such that ancilla qubits are |0>

4. Measurement

- Extract solution to a factor $\mathbb{N}^2$

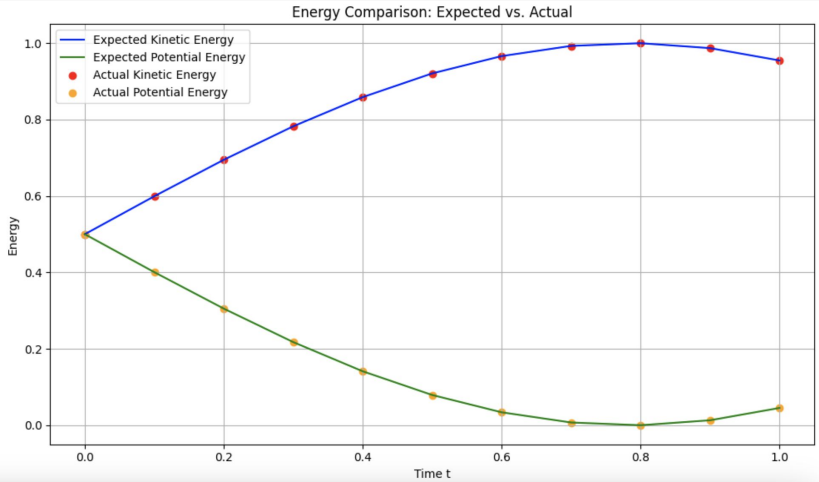
Exponential Speedup!

$O(N^3)$ vs. $O(k \log k \log N + \log N + k^2)$

Objective

LDE for Classical Harmonic Oscillator: $$y'' + \omega^2 y = 0, \quad y(0) = 1, \quad y'(0) = 1, \quad \omega = 1$$

Mapping Kinetic and Potential Energies: $$KE = \frac{1}{2}(y')^2 \qquad PE = \frac{1}{2}y^2$$



t	Kinetic expected	Kinetic actual	Potential expected	Potential actual)
0	0.5	0.5	0.5	0.5
0.1	0.599335	0.599335	0.400665	0.400665
0.2	0.694709	0.694709	0.305291	0.305291
0.3	0.782321	0.782321	0.217679	0.217679
0.4	0.858678	0.858678	0.141322	0.141322
0.5	0.920735	0.920735	0.0792645	0.0792645
0.6	0.96602	0.966019	0.0339805	0.0339804
0.7	0.992725	0.992723	0.00727514	0.00727498
0.8	0.999787	0.99978	0.000213198	0.000213276
0.9	0.986924	0.986907	0.0130762	0.0130777
1	0.954649	0.954611	0.0453513	0.0453579

```
kinetic accuracy (Mean):  99.9994056323564 %
potential accuracy (Mean):  99.99406978653249 %
```

Reference: Classiq. harmonic_oscillator.ipynb (quantum algorithm for solving linear differential equations). GitHub. https://github.com/Classiq/classiq-library/blob/main/community/paper_implementation_project/quantum_algo_for_solving_linear_differential_equations/harmonic_oscillator.ipynb :contentReference[oaicite:0]{index=0}

Varied Parameters

Interactive_plots.html

ω

Frequency
Tested Ranges: {1, 10}

y_0

Initial position
Tested Ranges:
{0, 0.5, 1, 1.5, 2, 2.5, 3, 10}

vy_0

Initial velocity
Tested Ranges:
{0, 0.5, 1, 1.5, 2, 2.5, 3, 10}

b1

Initial offset
Tested Ranges:
{0, 0.1, 0.2, ..., 1.0}

k

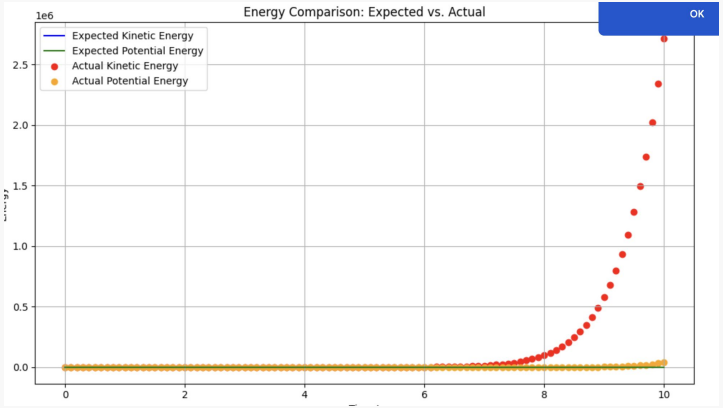
Order of expansion
Tested Ranges:
{1, 2, 3, ..., 15}

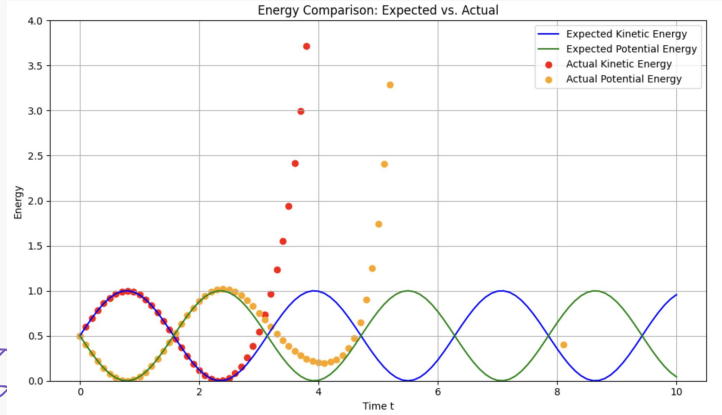
t

Time range
Tested Ranges:
{1, 10}

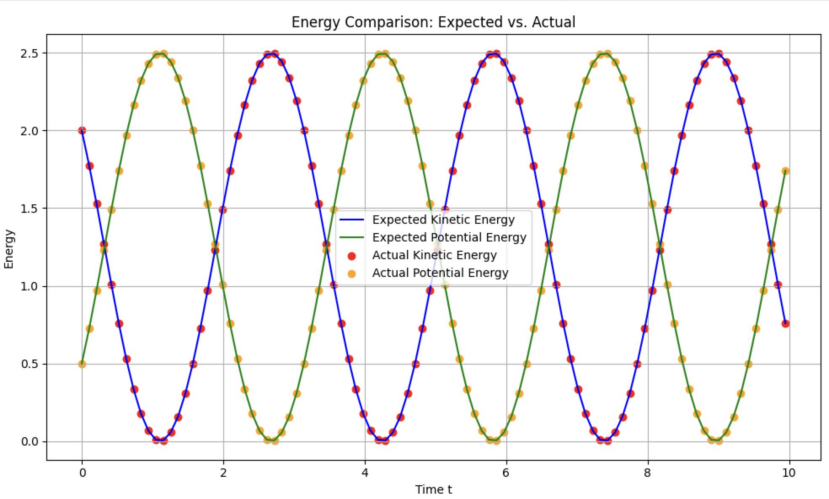
Changing Parameters: Time (t)

Originally: Solution blows up for high time





Noticing symmetry, we only need to simulate one-fourth of the time period of the oscillator ($\pi/2\omega$) to model energy overall

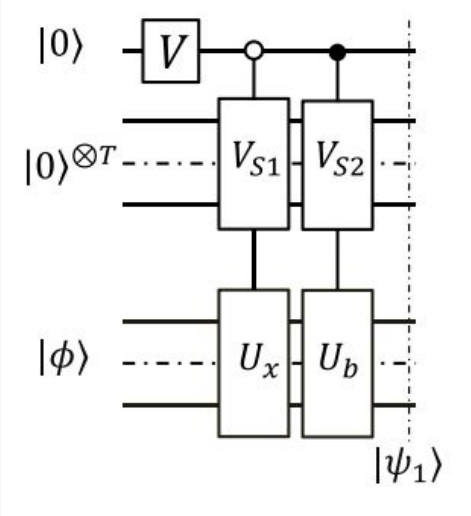


Changing Parameters: k (order of expansion)

Number of ancilla qubits: $T = \log_2(k+1)$ qubits

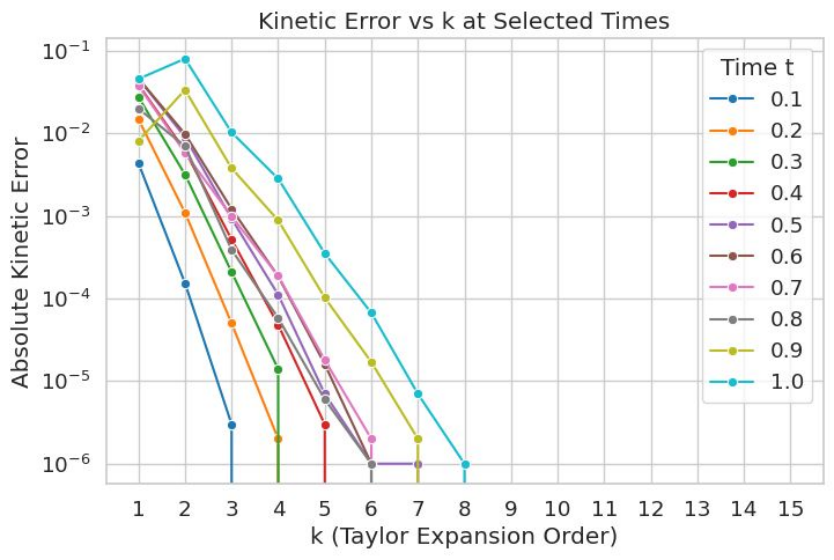
All ancilla registers prepared in $|0\rangle|0\rangle^T$

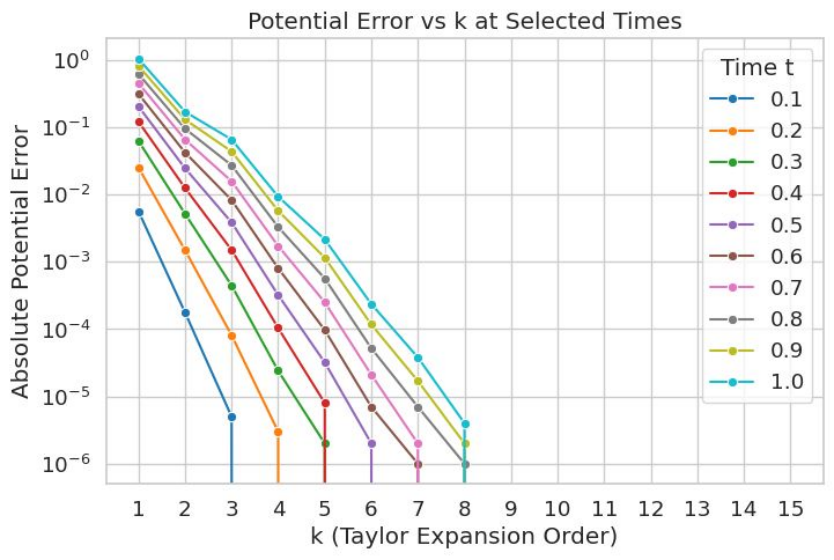
Auxiliary padding function in order to make the algorithm work for any k (not just k = 2^N–1)



Padded V_{S1} and V_{S2} to match ancilla register dimensions

Changing Parameters: k (order of expansion)





Larger k = smaller error since k corresponds with Taylor expansion accuracy

Changing ω (non-unitary matrix)

Varying ω changes matrix M: $\begin{bmatrix} 0 & 1 \\ -\omega^2 & 0 \end{bmatrix}$

M is no longer unitary; we must use the non-unitary case outlined in Appendix D

ω is frequency: larger ω = more oscillations = less accuracy

However, it requires 2k ancilla qubits (as compared to $\log_2(k+1)$ for the unitary case) which makes any meaningful computation too intensive

Changing initial conditions: y_0 & vy_0

- y_0 (initial position) determines wave amplitude
- vy_0 (initial velocity) is a boundary condition that determines wave amplitude, phase shift, and energy profile

Originally in the paper, $y_0 = 1$ and $vy_0 = 1$.

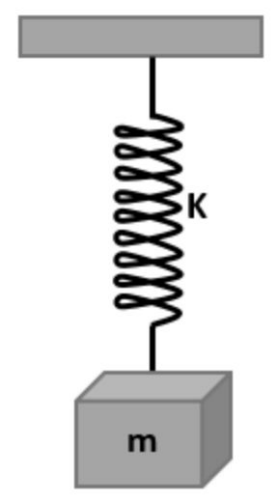
Limiting cases: Nothing happens when $y_0 = 0$ and $vy_0 = 0$.

To see how the graphs of energy, y, and y' vary as y_0 and vy_0 change, refer to the interactive plots html files on the attached GitHub Repository:

Interactive_plots.html

Changing Parameters: $b = [b_0, b_1]$ (initial offset)

- b_1 when nonzero is similar to adding a constant force to a system where equilibrium changes but not dynamics
  - Similarly to the role of gravity to a mass on a vertical spring



- b_0 changing shifts velocity which is aphysical and is always zero in real mechanical systems, so we did not bother varying b_0

To see how the graphs of energy, y, and y' vary as b_1 changes, refer to the interactive plots html files on the attached GitHub Repository:

[Interactive_plots.html]

Github Deliverables

interactive_plots/

- Interactive website to visualize harmonic oscillator energy, position, and velocity with varying parameters

Interactive Plot Codes

- Code for the interactive plots

Large Data Analysis Code for Varying Parameters

- Analysis over large data to determine overall trends over varying parameters

Longer Time Code

- Jupyter Notebook exploring how to simulate for longer time periods

Optimal k Plots

- Jupyter Notebook exploring arbitrary k's with plots

Conclusion

- Read: Quantum algorithm for solving linear differential equations: Theory and experiment by Tao Xin et al (2020)
- Varied parameters to analyze harmonic oscillator energy change
- Code to plot energies for longer times exploiting periodicity
- Code adjustment to handle arbitrary k (approximation orders) and k visualization plots
- Implemented non-unitary matrix [ω] algorithm
- Developed an interactive website to view energy, position, and velocity plots with varying parameters

Outlooks

- Multi-Particle Coupled Systems, calculating eigenvectors for such a system and animating it
- Damped Harmonic Oscillator
- Adding Anharmonicity & Performing Perturbation Analysis
- Hyperplane plot of all parameters varying, setting a threshold for mean (and every point's) accuracy to determine minimum k